1 Deploying OpenStack

VirtualBox Configuration

System config:

RAM: 8192 MB
Processor: 5 CPUs with PAE/NX enabled

Network:

Adapter 1: NAT (ip: 10.0.2.15)
Adapter 2: Host-only Adapter ('VirtualBox Host-Only Ethernet Adapter') with Promiscuous Mode set to "Allow All"
Target Static IP: 192.168.56.250

Storage: 217.57 GB (Dynamic Virtual Disk)

Operating System: Ubuntu 22.04 LTS Server

Installation Process:

During Ubuntu installation:

Storage Configuration:

1. Selected "use an entire disk"
2. Checked "set up this disk as an LVM group"
3. Confirmed destructive action to format the volume

Profile Configuration:

1. name: OpenStack Admin
2. server's name: openstack-main
3. Username: stack
4. Password: **********

SSH Configuration:

Checked [X] Install OpenSSH server

Additional Packages:

Skipped "Featured Server Snaps" to avoid conflicts

After the installation and successful reboot, logged in as stack user.

2 Network Configuration and SSH Access

Static IP with netplan:

1. by editing the configuration using this command:
   sudo nano /etc/netplan/00-installer-config.yaml
2. configure enp0s8 interface for static IP (192.168.56.250)
   sudo netplan apply
   Ip a

Result: confirmed that enp0s8 was successfully assigned IP 192.168.56.250

Establishing SSH connection with portforwarding:

command I used to SSH to my windows powershell:

```
ssh -p 2222 -L 0.0.0.0:8080:192.168.56.250:80 stack@127.0.0.1
```

This command performs :

1. connects to VM via Host Port 2222 (forwards to VM port 22)
2. binds local port 8080 to VM's port 80 for OpenStack Dashboard (Horizon) access at http://localhost:8080

RESULT : **successfully logged in with welcome message**

```
Welcome to Ubuntu 22.04.5 LTS

stack@openstack-main:~$
```

3. Installing Dependencies and Kolla-Ansible

Now I started with updating the system packages to make sure everything is up to date and there are no outdated dependencies present in the system.

System Update

in order to update the package list and upgrade the existing packages, I executed the following commands:

```
sudo apt update

sudo apt upgrade -y
```

this ensured that the system was running on the latest available updates.

Installing Required Dependencies

before installing Openstack using Kolla-ansible, certain system dependencies are required. These packages are necessary for python environment, SSL support and virtual environment creation.

Commands:

```
sudo apt install -y git python3-dev libffi-dev gcc libssl-dev python3-venv
```

Creating Python Virtual Environment

To avoid conflicts between system ython packages and openstack-related packages, I created a dedicated python virtual env.

1. first, I created a directory for the virtual environment:

   ```
   mkdir -p ~/kolla-venv
   ```

2. then I initialized the virtual environment:

   ```
   python3 -m venv ~/kolla-venv
   ```

3. after creating it, I activated the virtual environment using:

   ```
   source ~/kolla-venv/bin/activate
   ```

Once activated, all python-related installations were done inside this isolated environment.

Installing Ansible and Kolla-Ansible

1. after activating the virtual environment, I updated pip to the latest version:

   ```
   pip install -U pip
   ```

2. then installed Ansible Core with a specific version range to ensure compatibility with Kolla-Ansible:

   ```
   pip install "ansible-core>=2.15,<2.16.99"
   ```

3. next, installed Kolla-Ansible version 17.1.0:

   ```
   pip install kolla-ansible==17.1.0
   ```

Setting Up Configuration Directory

1. once Kolla-Ansible was installed successfully, I created the configuration directory required by Kolla.

   ```
   sudo mkdir -p /etc/kolla
   ```

2. then I changed the ownership of this directory to the current user to avoid permission issues:

   ```
   sudo chown $USER:$USER /etc/kolla
   ```

3. after that, I copied the example configuration files provided by Kolla-Ansible:

   ```
   cp -r ~/kolla-venv/share/kolla-ansible/etc_examples/kolla/* /etc/kolla/
   ```

   ```
   cp ~/kolla-venv/share/kolla-ansible/ansible/inventory/all-in-one
   ```

Password generation:

Command:

```
kolla-genpwd
```

the above command automatically creates secure passwords and stores them in the appropriate configuration files.

Configuring globals.yml

After generating the passwords, I edited the global configuration file to define the deployment parameters:

```
nano /etc/kolla/globals.yml
```

following configuration entries were added in the file:

```
kolla_base_distro: "ubuntu"
openstack_release: "2023.2"
kolla_internal_vip_address: "192.168.56.250"
network_interface: "enp0s8"
neutron_external_interface: "enp0s8"
enable_horizon: "yes"
enable_swift: "yes"
enable_cinder: "yes"
enable_cinder_backend_lvm: "yes"
```

these changes defined the base distribution, openstack release version, network interfaces, virtual ip, and enabled the required services such as Horizon, swift, and cinder.

4. Configuring Storage and Bootstrap

After completing the Kolla configuration, I moved forward with setting up storage for Cinder and Swift. Since this was a single-node lab setup, I created virtual loopback disks to simulate separate storage devices.

Creating Virtual Loopback Drives for Cinder and Swift

1. first, I created directories to store the disk image files:

   sudo mkdir -p /var/lib/cinder_data /var/lib/swift_data

2. then created two 10GB disk image files, one for Cinder volumes and one for Swift storage:

   sudo fallocate -l 10G /var/lib/cinder_data/cinder-volumes.img

   sudo fallocate -l 10G /var/lib/swift_data/swift.img

3. after creating the disk files, I attached them to loopback devices:

   sudo losetup /dev/loop10 /var/lib/cinder_data/cinder-volumes.img

   sudo losetup /dev/loop11 /var/lib/swift_data/swift.img

this allowed the system to treat the image files as block devices.

Initializing Cinder Volume Group

1. to prepare storage for Cinder, I first created a physical volume:

   sudo pvcreate /dev/loop10

2. then I created a volume group named cinder-volumes:

   sudo vgcreate cinder-volumes /dev/loop10

volume group would be used by Cinder's LVM backend for block storage.

Formatting Swift Disk

Next, I prepared the Swift storage disk.

1. since swift works efficiently with the XFS filesystem, I formatted the disk as follows:

   sudo mkfs.xfs -f -L d1 /dev/loop11

2. I created a mount point:

   sudo mkdir -p /srv/node/d1

3. mounted the filesystem using the label:

   sudo mount -t xfs -L d1 /srv/node/d1

This mounted the Swift storage device at /srv/node/d1.

Building Swift Rings Manually

ERROR 1 ENCOUNTERED:

the command swift-ring-builder was not found.

Resolution:

This occurred because swift utilities were not installed inside the virtual environment. I installed the required packages using:

pip install python-swiftclient python-openstackclient

pip install swift

Creating Swift Rings

After installing the required utilities, I manually created the Swift ring configuration.

First, I created the configuration directory:

mkdir -p /etc/kolla/config/swift

cd /etc/kolla/config/swift

Building Object Ring:

swift-ring-builder object.builder create 10 1 1

swift-ring-builder object.builder add r1z1-192.168.56.250:6000/d1 100

swift-ring-builder object.builder rebalance

Building Account Ring:

swift-ring-builder account.builder create 10 1 1

swift-ring-builder account.builder add r1z1-192.168.56.250:6001/d1 100

swift-ring-builder account.builder rebalance

Building Container Ring:

swift-ring-builder container.builder create 10 1 1

swift-ring-builder container.builder add r1z1-192.168.56.250:6002/d1 100

swift-ring-builder container.builder rebalance

each ring was configured with region 1 zone 1, and appropriate port numbers for object (6000), account (6001), and container (6002) services.

Bootstrap Server Attempt

After completing the storage configuration, I proceeded with bootstrapping the servers:

kolla-ansible -i all-in-one bootstrap-servers

ERROR 2 ENCOUNTERED:

ansible version mismatch. The system had Ansible 2.16 installed, but kolla-ansible 2023.2 requires version 2.14 or 2.15.

Resolution:

1. first, I removed the incompatible version:

pip uninstall -y ansible ansible-core

2. then I installed a compatible version:

pip install ansible-core==2.15.9

ERROR 3 ENCOUNTERED:

missing openstack.kolla.baremetal role

Resolution:

To resolve this, I manually cloned the required ansible collection.

```
mkdir -p ~/.ansible/collections/ansible_collections/openstack
cd ~/.ansible/collections/ansible_collections/openstack
git clone https://opendev.org/openstack/ansible-collection-kolla kolla
```

Successful Bootstrap

after resolving all the above errors, I ran the bootstrap command again:

```
kolla-ansible -i all-in-one bootstrap-servers
```

This time the bootstrap process completed successfully…..

5. Pre-Deployment Checks

After successfully completing the bootstrap process, I proceeded with running pre-deployment checks to verify that the system configuration was correct before actual deployment.

Running Prechecks

to perform the validation checks, I executed the following command:

```
kolla-ansible -i all-in-one prechecks
```

ERROR 1 ENCOUNTERED:

could not load "ipaddr" plugin.

error message indicated:

invalid plugin FQCN ... unable to locate collection ansible.netcommon

this error occurred because the required Ansible collection ansible.netcommon was not installed in the environment.

Resolution:

I installed the missing collection using:

```
ansible-galaxy collection install ansible.netcommon
```

ERROR 2 ENCOUNTERED:

After installing the collection, a warning was displayed:

collection ansible.netcommon does not support Ansible version 2.15.9.

this indicated a compatibility issue between the installed collection version and the current ansible version.

Resolution:

1. to fix this issue, installed the required python dependency:

   pip install netaddr

2. Then I installed specific compatible versions of the required Ansible collections:

   ansible-galaxy collection install ansible.netcommon:==5.1.2 ansible.utils:==2.10.3 --force

this ensured compatibility with Ansible Core version 2.15.9.

Successful Prechecks

after resolving the above issues, I ran the prechecks command again:

kolla-ansible -i all-in-one prechecks

Result:

localhost : ok=120 failed=0 .....

pre-deployment validation completed successfully..!!!

6. Deployment and Error Handling

After successful completion of prechecks, I proceeded with the actual deployment of openstack services.

Initial Deployment Attempt

to begin the deployment process, I executed the following command:

kolla-ansible -i all-in-one deploy

ERROR 1 ENCOUNTERED:

during the deployment process, the SSH session suddenly terminated with the following message:

client_loop: send disconnect: Connection reset

Issue Identified:

the VM ran out of system resources during neutron container deployment. As a result, SSH connection was terminated

Action Taken:

I accessed the Virtualbox console and forced reboot of the virtual machine to restore system stability.

Result: After rebooting, the deployment process was resumed.

Network Recovery and Final Deployment

after rebooting the virtual machine due to the resource issue, I observed that the network interface was not working properly. The interface enp0s8 was captured by Open vSwitch bridges and was not operating normally.

Network Interface Recovery After Reboot

To verify the interface status, I checked the ip a, specifically enp0s8:it was observed that the interface was "enslaved" to ovs-system and the static IP address was missing.

Removing Stuck Network Bridges

1. To resolve this issue, I first stopped the Docker service to avoid interference:
2. then I removed the Open vSwitch kernel module:
3. after removing the module, I manually freed the network interface:

```
sudo ip link set enp0s8 nomaster

sudo ip link set enp0s8 down

sudo ip link set enp0s8 up
```

This detached the interface from any bridge associations and re-initialized it.

Verifying Network Recovery

To confirm whether the interface was restored properly, I used he command ip a :

and it validated that **:** inet 192.168.56.250/24 ... scope global dynamic enp0s8

this confirmed that the IP address was successfully restored and the network interface is functioning correctly.

Restoring Loopback Storage Devices

after reboot, the loopback devices were not automatically attached, so I manually restored them.

1. restoring Cinder loopback device

```
sudo losetup /dev/loop10 /var/lib/cinder_data/cinder-volumes.img
```

2. restoring Swift loopback device

```
sudo losetup /dev/loop11 /var/lib/swift_data/swift.img
```

3. then I activated the Cinder volume group

```
sudo vgchange -a y cinder-volumes
```

4. finally, I mounted the Swift storage again

```
sudo mount -t xfs -L d1 /srv/node/d1
```

This ensured that both cinder and swift storage were available before restarting deployment.

Final Successful Deployment

After restoring the network and storage configuration, I ran the deployment command again:

```
kolla-ansible -i all-in-one deploy
```

this time i got, SUCCESS!!! with failed=0…!!!

8. Post-Deployment

After successful deployment, i moved to post-deployment configuration and verification of services.

Generating Admin Credentials

1. To generate the required post-deployment configuration files, I executed:

   kolla-ansible -i all-in-one post-deploy

   Result: Post deploy was successful and with failed=0

2. installed the OpenStack command-line client:

   pip install python-openstackclient

3. Then I loaded the admin credentials:

   source /etc/kolla/admin-openrc.sh

4. To verify that the cloud was operational, i ran:

   openstack token issue command

result: the token was successfully issued, which confirmed that the openstack services were running correctly.

503 Service Unavailable Error

When I attempted to access the Horizon dashboard at:

http://localhost:8080

I received a 503 Service Unavailable error.

Investigating HAProxy Logs

1. To investigate the issue, I checked the HAProxy container logs:

   sudo docker logs --tail 20 haproxy

   critical errors were observed:

   Server horizon/horizon1 is DOWN, timeout

   Server mariadb_back/mariadb is DOWN, timeout

This indicated that HAProxy could not communicate with backend services.

Root Cause Discovery

I inspected the network configuration with command ip a

Error found:

the virtual IP 192.168.56.250 was present, but the Base IP 192.168.56.103 was missing from the interface.

HAProxy listens on the VIP address but forwards requests to the hostname openstack, which resolves to the Base IP. Since the Base IP was not configured on the interface, all forwarded traffic was dropped.

Resolution:

To fix the issue,

1. I manually added the missing Base IP to the network interface:

   sudo ip addr add 192.168.56.103/24 dev enp0s8

2. Then I ensured the interface was active:

   sudo ip link set dev enp0s8 up

3. after restoring the correct IP configuration, I restarted the HAProxy container:

   sudo docker restart haproxy

Verification

To confirm the fix, I checked the HAProxy logs again, I observed the results, which were:

Server horizon/horizon1 is UP

this confirmed that backend services were now reachable.

Dashboard Access

URL:

http://localhost:8080

login details:

username: **admin**

password: Retrieved using: **grep OS_PASSWORD /etc/kolla/admin-openrc.sh**

Result: The login page appeared and the Horizon dashboard was fully functional.

Quick Access Commands

To begin any new session,:

source ~/kolla-venv/bin/activate

source /etc/kolla/admin-openrc.sh

these commands activated the python virtual environment(kolla-venv) and loaded openstack admin credentials to access the services in CLI.

9. Swift Implementation and Configuration

After confirming that the openstack deployment was fully operational, I proceeded with configuring swift for object storage and integrating it with Nextcloud.

Creating Dedicated Nextcloud Storage Project

1. to isolate Nextcloud storage from the default admin project, I created a dedicated openstack project:

   openstack project create --domain default --description "Nextcloud Storage" service_storage

2. created a separate user for Nextcloud:

   openstack user create --domain default --password nextcloud123 nextcloud_user

3. then I assigned the member role to this user within the newly created project:

   openstack role add --project service_storage --user nextcloud_user member

this ensured that Nextcloud would only have access to its own storage project.

Setting Environment Variables for Nextcloud User

To operate as the nextcloud_user, I exported the following environment variables:

```
export OS_USERNAME=nextcloud_user
export OS_PASSWORD=nextcloud123
export OS_PROJECT_NAME=service_storage
export OS_PROJECT_DOMAIN_ID=default
export OS_USER_DOMAIN_ID=default
export OS_AUTH_URL=http://192.168.56.250:5000/v3
```

this allowed all subsequent CLI commands to execute within the service_storage project context.

Creating Nextcloud Container

I attempted to create a swift container:

openstack container create nextcloud_data

ERROR: Service Unavailable (HTTP 503)

the container creation failed with a 503 error, indicating that swift services were not functioning correctly.

Troubleshooting Swift Service

to verify the swift endpoint registration, I ran:

openstack endpoint list --service object-store --interface public

output confirmed the correct endpoint:

http://192.168.56.250:8080/v1/AUTH_%(tenant_id)s

1. checked whether all Swift containers were running:
   All Swift containers were in running state.

2. Then examined the Swift proxy logs:

   docker logs swift_proxy_server

It was observed that Swift failed to create the account namespace.

To ensure the storage directory existed:

ls -ld /srv/node/d1

The directory was present.

to verify whether any data had been written:

ls -R /srv/node/d1

The directory was empty, which confirmed that Swift never reached the write stage.

i checked for ring builder files inside the container:

docker exec -it swift_proxy_server swift-ring-builder /etc/kolla/swift/account.builder

the ring builder file did not exist inside the container.

however, on the host system:

ls /etc/kolla/config/swift/*.gz

The ring files were present:

1. account.ring.gz
2. container.ring.gz
3. object.ring.gz

This confirmed that ring files existed but were not properly mounted inside the container.

Copying Ring Files to Correct Location

1. to resolve this issue, I manually copied the ring files:

   sudo cp /etc/kolla/config/swift/*.ring.gz /etc/kolla/swift/

2. then I forced Swift service reconfiguration:

   kolla-ansible -i all-in-one reconfigure -t swift

3. since containers run with non-root UID, I corrected ownership:

   sudo chown -R stack:stack /etc/kolla/swift/

4. to confirm visibility inside the container:

   docker exec -it swift_proxy_server ls -l /etc/kolla/swift/

   Result: The ring files were now visible inside the container.

Investigating Mount Check Errors

1. ichecked the Swift object server logs(-- tail 20):

   found: Errors related to "mount checks" were found, indicating that swift was rejecting the storage device.

2. verify that hash paths were configured correctly:

   sudo grep "swift_hash_path" /etc/kolla/swift/swift.conf

   result: the hash configuration was confirmed to be present.

Disabling Mount Check Validation

1. since loopback devices were being used (not physical disks), Swift mount validation was failing. To resolve this, I disabled mount checks:

   sudo sed -i 's/mount_check = true/mount_check = false/g' /etc/kolla/swift/*.conf

2. to ensure all configuration files were updated:

   sudo find /etc/kolla/swift/ -name "*.conf" -exec sed -i 's/mount_check = true/mount_check = false/g' {} +

3. then I regenerated the Swift configuration:

   kolla-ansible -i all-in-one genconfig -t swift

4. to confirm that all required ports were listening:

   sudo ss -tulpn | grep -E '8080|6000|6001|6002'

fianlly, I restarted the Swift proxy container

Successful Container Creation

After resolving all issues, I retried container creation:

openstack container create my_first_container

output: the container was created with name : my_first_container

this confirmed that the container was created successfully and working correctly by uploading the data files….

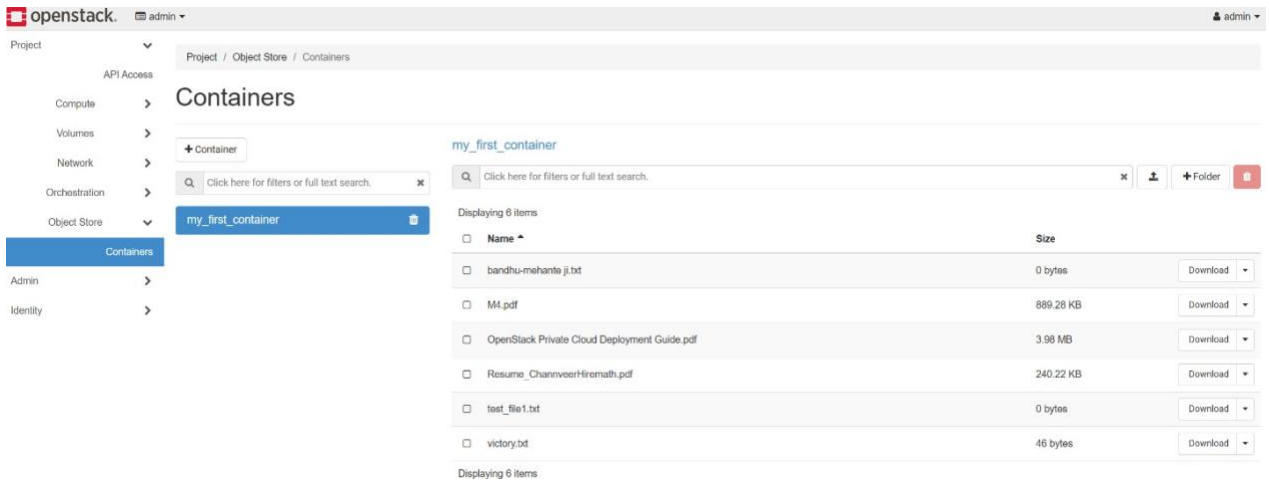


Fig:swift container

10. Validating the Cloud: Launching the First Instance

After completing storage and Swift configuration, I proceeded to validate the overall cloud functionality by launching the first virtual machine instance. This step was performed to confirm that compute (nova), networking (neutron), and image (glance) services were working correctly.

Downloading and Uploading Cirros Image

For testing purposes, I used the lightweight Cirros cloud image (approx 15MB).

1. first I downloaded the image:

   wget http://download.cirros-cloud.net/0.6.2/cirros-0.6.2-x86_64-disk.img

2. next I uploaded the image to the Glance image service:

   ```
   openstack image create "cirros" \
     --file cirros-0.6.2-x86_64-disk.img \
     --disk-format qcow2 \
     --container-format bare \
     --public
   ```

this made the image available for instance creation.

Creating Flavor

Since the environment had limited resources, I created a small flavor:

```
openstack flavor create --id 1 --vcpus 1 --ram 512 --disk 1 m1.tiny
```

This flavor defines:

1. 1 vCPU
2. 512 MB RAM
3. 1 GB Disk

Creating Network Infrastructure

1. to allow instances to communicate, I created a private network:

   ```
   openstack network create demo-net
   ```

2. then I created a subnet with DNS configuration:

   ```
   openstack subnet create --network demo-net \
     --subnet-range 10.0.0.0/24 \
     --gateway 10.0.0.1 \
    --dns-nameserver 8.8.8.8 \
     demo-subnet
   ```

this configured a private network range for instances.

Launching First Instance

To launch the first virtual machine:

openstack server create --image cirros --flavor m1.tiny --network demo-net Viva-test-vm

Troubleshooting BUILD Status

After issuing the command, I checked the instance status:

openstack server list

the VM was stuck in BUILD status.

Investigation

Further investigation showed that the Nova compute service was marked as "unhealthy". This was mainly due to the earlier network recovery process, which affected container service.

Resolution

1. to resolve the issue, I restarted the Nova compute container:

   sudo docker restart nova_compute

2. then I deleted the stuck instance:

   openstack server delete Viva-test-VM

3. after that, I launched a new instance:

   openstack server create --image cirros --flavor m1.tiny --network demo-net Viva-test-VM

Successful Instance Launch

to confirm the instance status:

openstack server list

output: The instance successfully reached **ACTIVE** state.

SUCCESS! compute , network and image services were confirmed to be functioning correctly.

Later, meanwhile on time I cleared the not required data files and some programms so that my RAM get freed up,then after I launched multiple instances with even medium falvor for The Windows image named as MY-Windows -server….

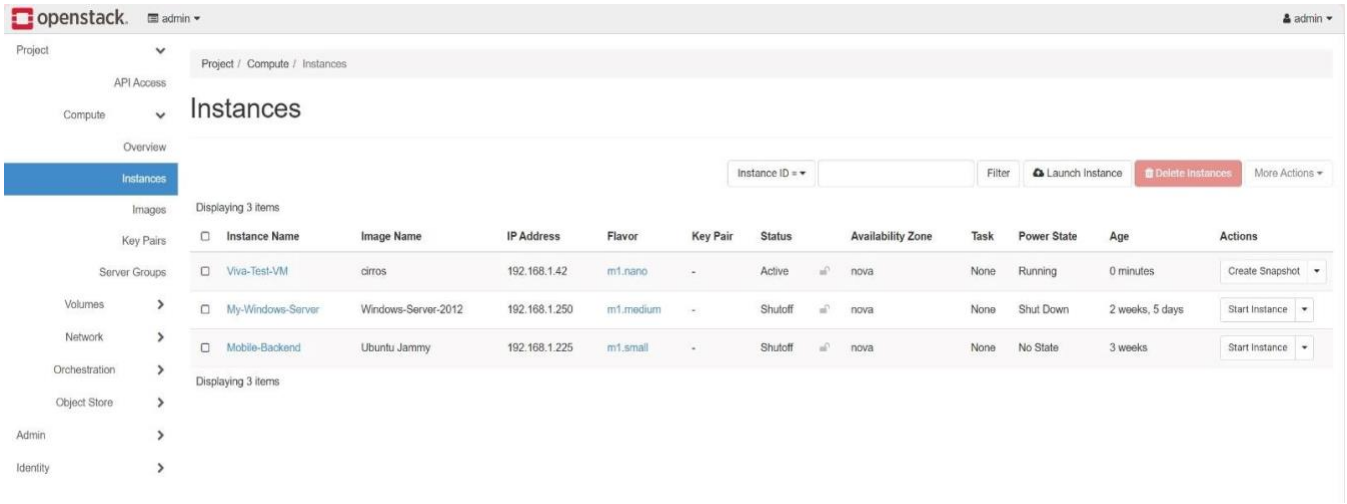


fig:The instances

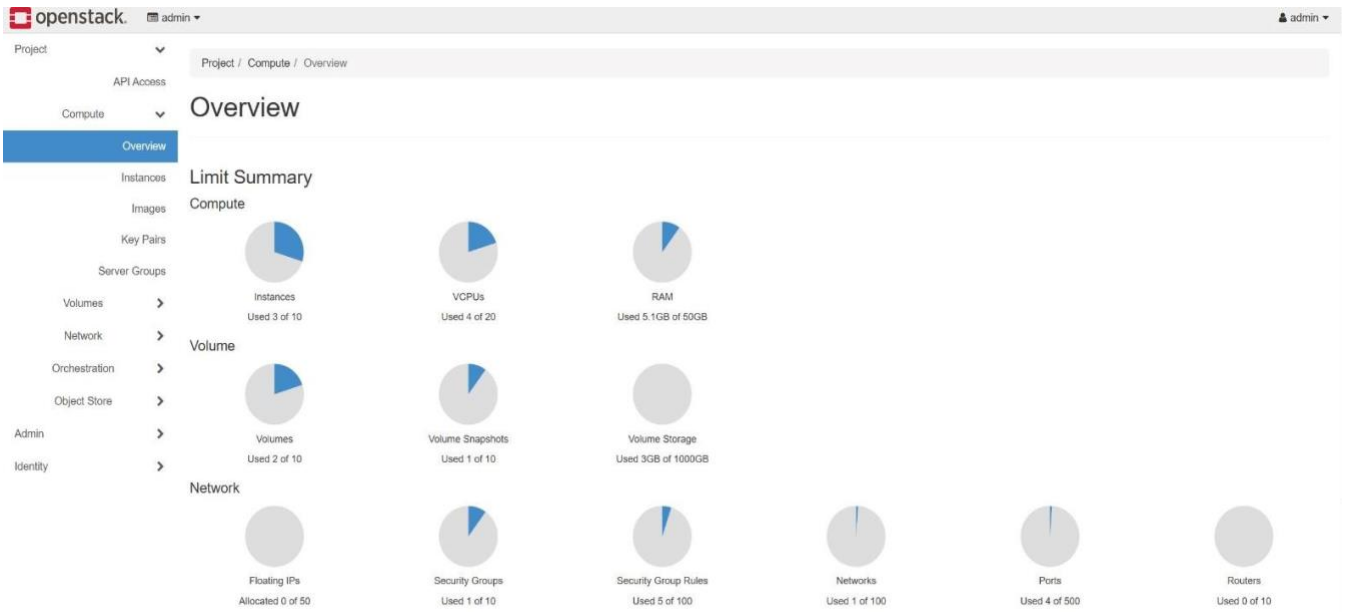


Fig:Overview(Dashboard)

11. Automation using Python (Hybrid Cloud Sync)

Automation using Python (Hybrid Cloud Sync)

After verifying that Swift storage and instance launch were working correctly, I implemented an automation mechanism on Windows to synchronize local files with OpenStack Swift object storage. This created a basic hybrid cloud synchronization model.

Installing Python Libraries on Windows

To enable authentication and communication with Swift, I installed the required Python libraries:

pip install python-swiftclient keystoneauth1

these libraries allow the script to authenticate using Keystone (v3) and upload files to Swift.

Folder Structure Setup

on the Windows Desktop, I created the following folder structure:

Cloud_Upload/ → Watch folder for new files
Cloud_Upload/Sent/ → Archive folder for successfully uploaded files

the script continuously monitors the Cloud_Upload folder.

Python Automation Script

Configuration Section:

OS_AUTH_URL = 'http://192.168.56.250:5000/v3'

OS_USERNAME = 'admin'

OS_PASSWORD = '**********'

CONTAINER_NAME = 'my_first_container'

WATCH_FOLDER = r"C:\Users\Channveerswami H\OneDrive\Desktop\Cloud_Upload"

SENT_FOLDER = r"C:\Users\Channveerswami H\OneDrive\Desktop\Cloud_Upload\Sent"

Running Script Silently in Background

To run the script without opening a console window:

Rename script extension

background_sync.py → background_sync.pyw

After renaming, I double-clicked the file to execute it silently.

Verification:
checked Task manager and confirmed that pythonw.exe was running in the background.

Verification Test

procedure:

1. dropped file_test2.txt into Cloud_Upload folder
2. waited 4 minutes (as defined in script sleep interval)
3. file was automatically moved to sent folder
4. verified in OpenStack Dashboard that the file appeared in container my_first_container

this confirmed that automation was functioning correctly.

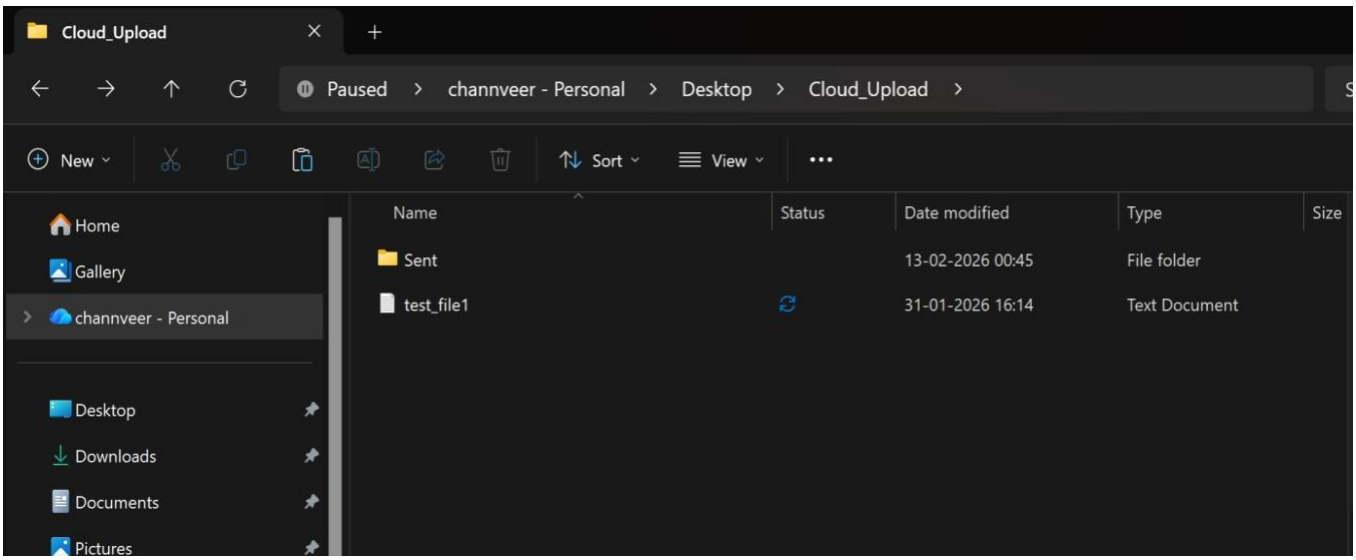


Fig: watch-dog folder (Cloud_upload)

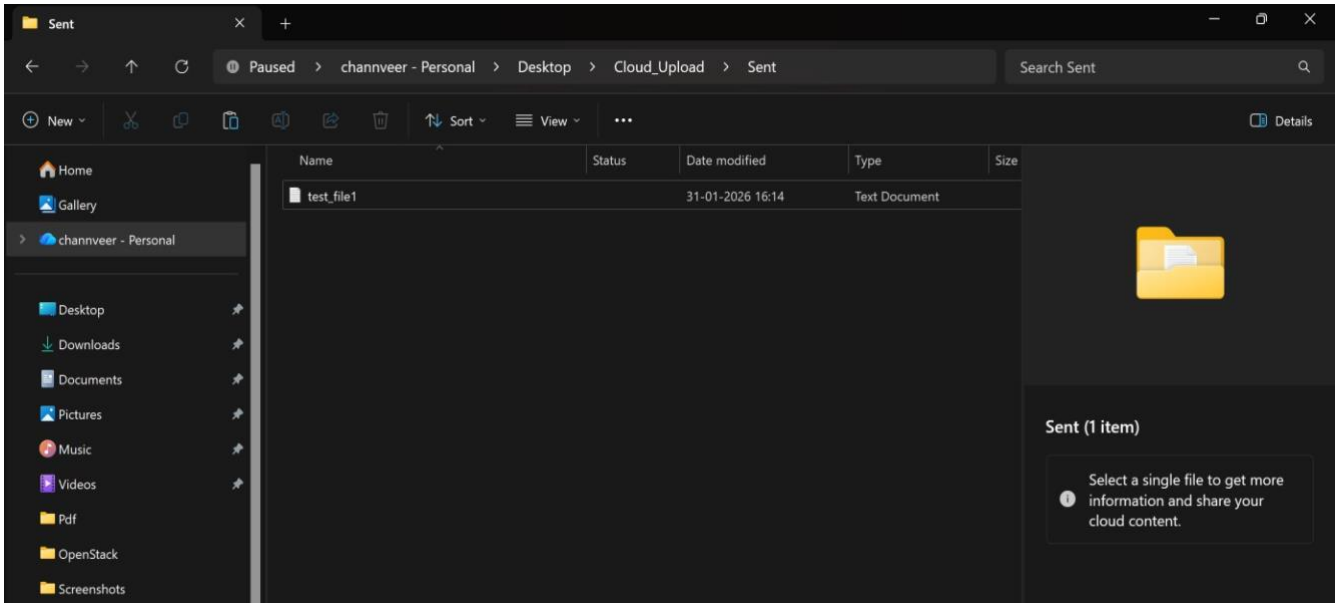


Fig: Sent folder(where all dropped data files are stored after when they are sent to cloud)

Pseudocode – Automation Logic

Step 1:
Start the background_sync.pyw file on Windows.

Step 2:
load the configuration parameters:

3. openStack Authentication URL
4. username and Password
5. project Name
6. swift Container Name
7. watch Folder Path
8. sent Folder Path

Step 3:
Establish connection with OpenStack:

9. authenticate using Keystone v3
10. create authenticated session
11. initialize Swift client connection

Step 4:
Check whether the Watch Folder exists.

12. if not found → display error and stop.

Step 5:
scan the Watch Folder for new files.

Step 6:
If no new files are found:

13. display message "No new files to upload."
14. wait for defined time interval (4 minutes).
15. repeat from Step 5.

Step 7:
If files are found:

16. connect to Swift container
17. ensure container exists (create if not present)

Step 8:
For each file in Watch Folder:

18. open file in binary mode
19. upload file to Swift Object Storage
20. close file

Step 9:
After successful upload:

21. check if Sent folder exists
22. if not, create Sent folder
23. move uploaded file from Watch Folder to **sent** folder

Step 10:
If any error occurs during upload:

>display error message

>continue to next file

Step 11:
wait 240 seconds (4 mins).

Step 12:
Repeat the process continuously in background.

For source code, visit : https://github.com/channveerswamiHiremath/Openstack-automation-script.git

12. Installing Nextcloud (SaaS Layer)

After validating the IaaS layer (Compute, Network, Storage), I proceeded with installing Nextcloud to act as the SaaS layer on top of openstack infrastructure.

Enabling IP Forwarding

Before launching the container, I enabled IPv4 forwarding to allow proper container networking:

```
sudo sysctl -w net.ipv4.ip_forward=1
```

this ensured that Docker container traffic could be routed correctly.

Initial Container Attempt (Failed)

1. i attempted to launch the Nextcloud container using:

   ```
   docker run -d --name nextcloud-container --restart always -p 8082:80 nextcloud
   ```

2. to verify container status:

   ```
   docker ps
   ```

Issue Observed:
The PORTS column was empty, indicating a networking conflict. The container was running but port mapping was assigned.This was caused by conflicts between Docker's default bridge network and openstack networking components.

Creating Custom Docker Network

1. to resolve the networking conflict, I removed the failed container:
2. then I created a custom Docker network to isolate the nextcloud container from openstack managed networking:

   ```
   sudo docker network create nextcloud-net
   ```

   t

   his allowed the container to run on a separate bridge.

Successful Container Launch

3. after creating the custom network, I relaunched nextcloud:

```
sudo docker run -d \
  --name nextcloud-container \
 --network nextcloud-net \
  --restart always \
   -p 8082:80 \
    nextcloud
```

to verify correct port mapping

Output showed:

0.0.0.0:8082->80/tcp

This confirmed that the container was successfully mapped to port 8082.

Initial Nextcloud Setup

Access URL:

http://192.168.56.250:8082

configuration during first login:

1. username: ad***
2. password: ope******
3. database: SQLite

After completing the setup process, the Nextcloud dashboard loaded successfully.

RESULT:

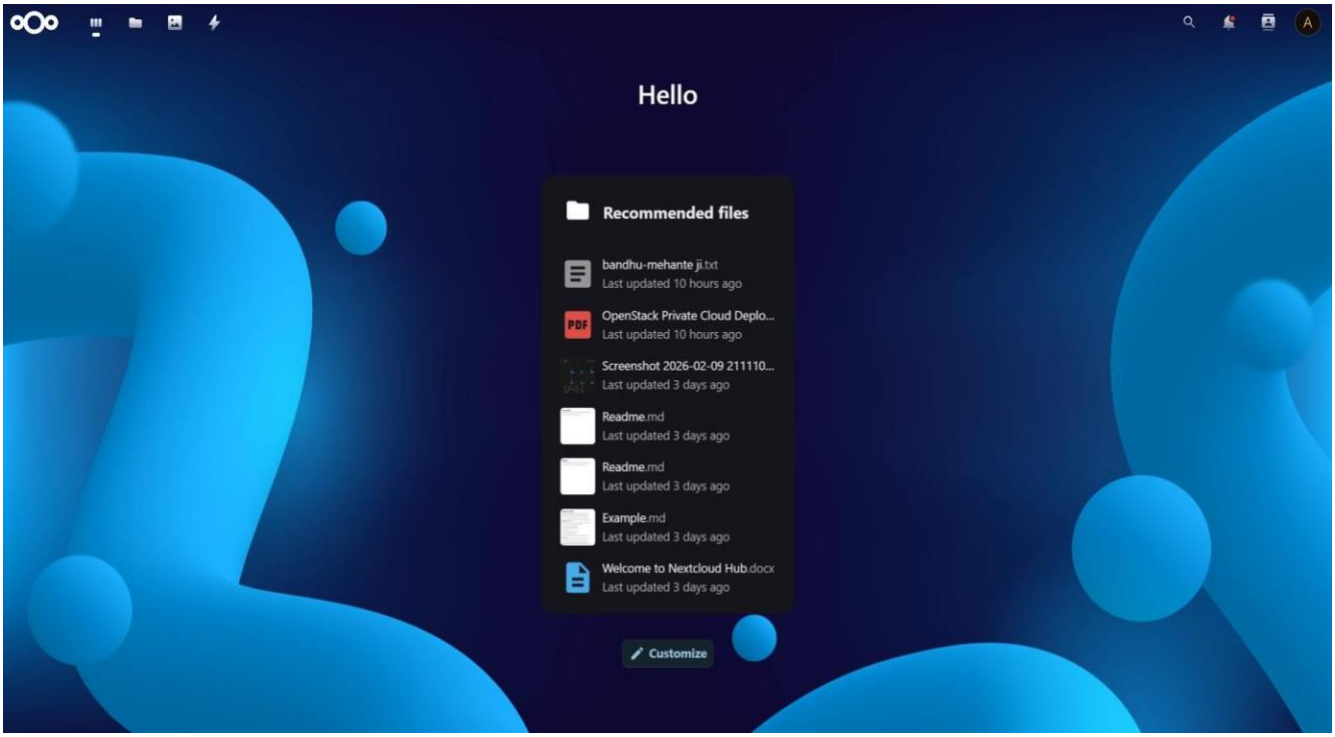


Fig:Dashboard

13. Integrating Nextcloud with OpenStack Swift

After successfully installing Nextcloud (SaaS layer), I proceeded to integrate it with OpenStack Swift object storage (IaaS backend).

Enabling External Storage App

I logged into the Nextcloud dashboard.

Navigation steps:

1. clicke on user icon

2. click >selected Apps

3. enabled the app "External storage support"

This app allows nextcloud to connect to external storage systems such as openstack swift.

Configuring Swift Connection

Then I navigated to:

Administration Settings → External Storage

I added a new external storage entry with the following parameters:

1. folder Name: MyOpenStack
2. external Storage Type: OpenStack Object Storage
3. authentication: OpenStack (Keystone v3)
4. region: RegionOne
5. bucket: my_first_container
6. Identity Endpoint URL: http://192.168.56.250:5000/v3
7. domain: default
8. tenant: admin
9. username: admin
10. password: op******
11. Available for: All people (Checked)

After entering the configuration details, i click save.

Verification

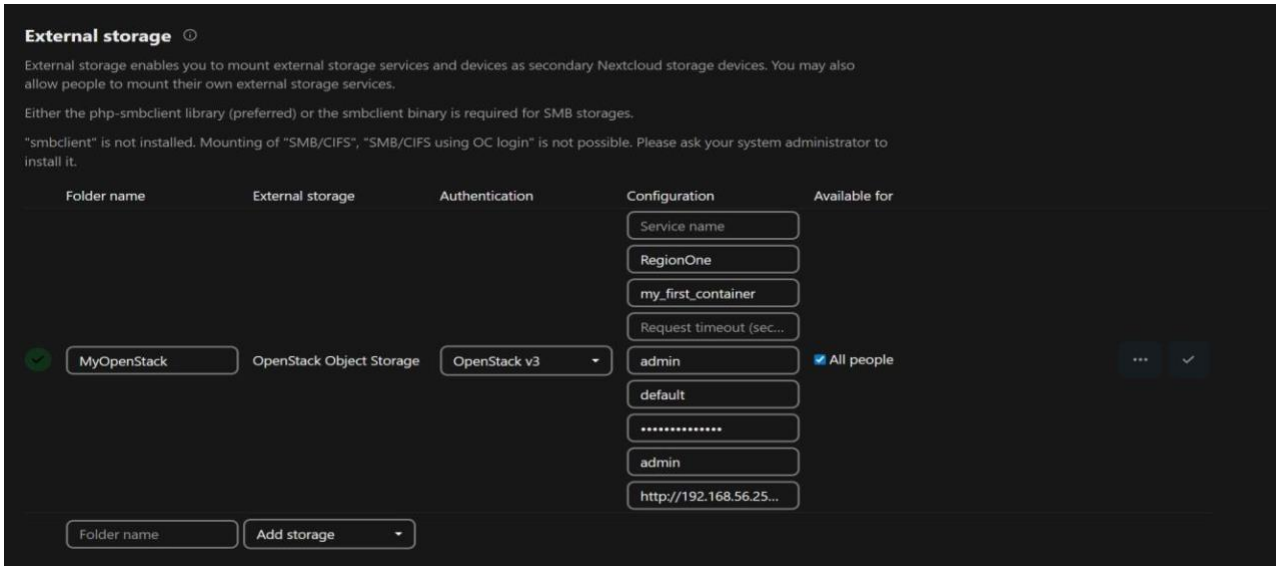


Fig: External storage configure

A green circle appeared next to the configuration entry, confirming that the connection to Swift was successful.

The folder MyOpenStack was visible in Files section.

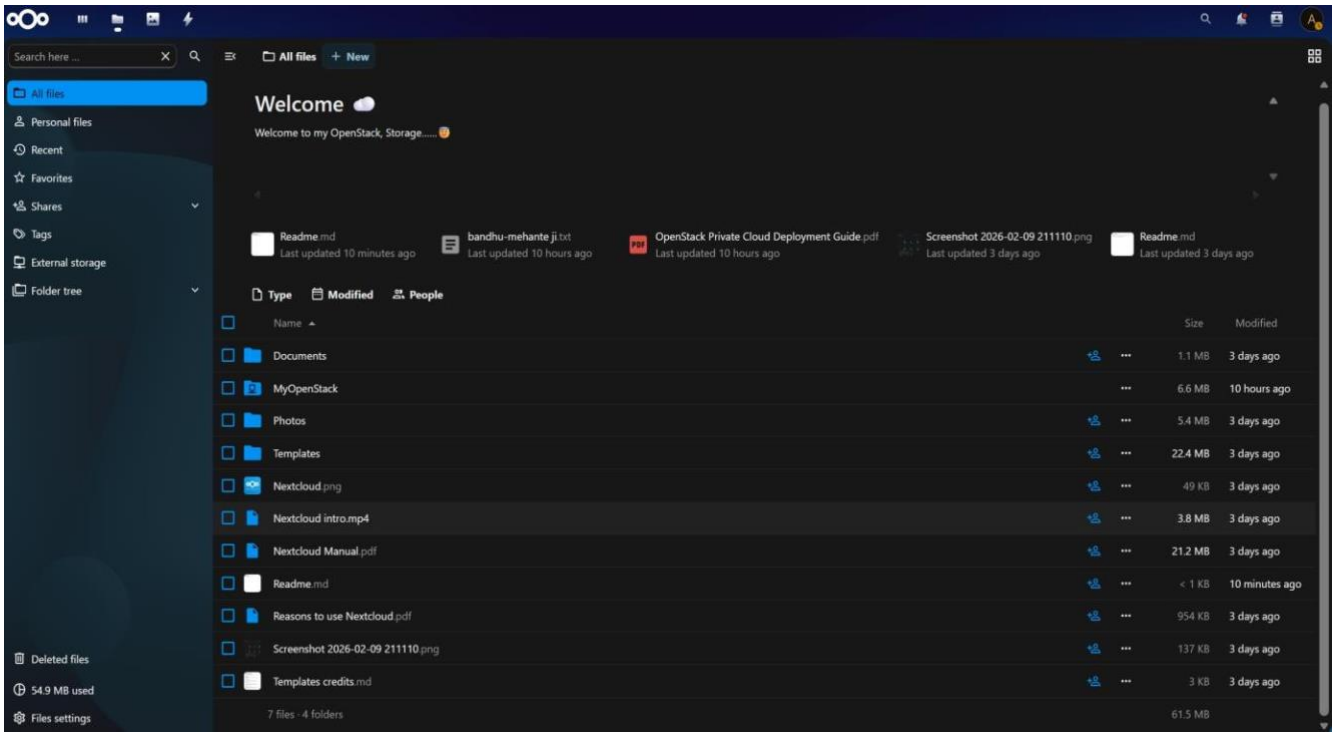


Fig: Files section

To verify functionality:

1. uploaded a pdf M4.jpeg inside the MyOpenStack(nextcloud) folder.
2. opened openstack dashboard.
3. navigated to Object Store → Containers.
4. opened container my_first_container.

The file appeared instantly in swift storage.

This confirmed that file uploads from Nextcloud were being stored in openstack swift.

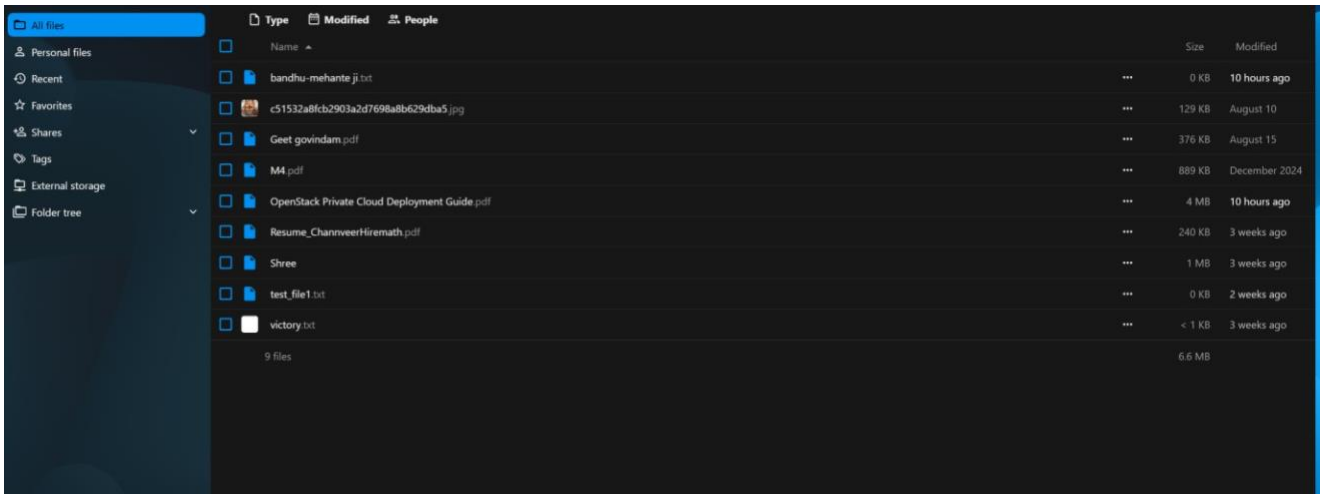


Fig: we can observer M4.pdf

14. Global Access with Tailscale (VPN)

After successfully integrating Nextcloud with OpenStack Swift, I configured secure global access using Tailscale VPN. This allowed remote users to securely connect to the private cloud from anywhere.

Installing Tailscale on Server

1. To install Tailscale on the OpenStack host machine:

   curl -fsSL https://tailscale.com/install.sh | sh

2. Then I authenticated the machine with my Tailscale account:

   sudo tailscale up

A login link was generated. I opened the link in a browser and authenticated using my Google account.

3. To retrieve the assigned VPN IP address:

   tailscale ip -4

4. VPN IP Assigned:

   100.104.231.107

This IP address allows secure VPN-based access to the private cloud.

Whitelisting Tailscale Network in Nextcloud

To allow connections from the VPN network, I updated Nextcloud's trusted domains configuration:

sudo docker exec --user www-data nextcloud-container php occ config:system:set trusted_domains 10 --value=*

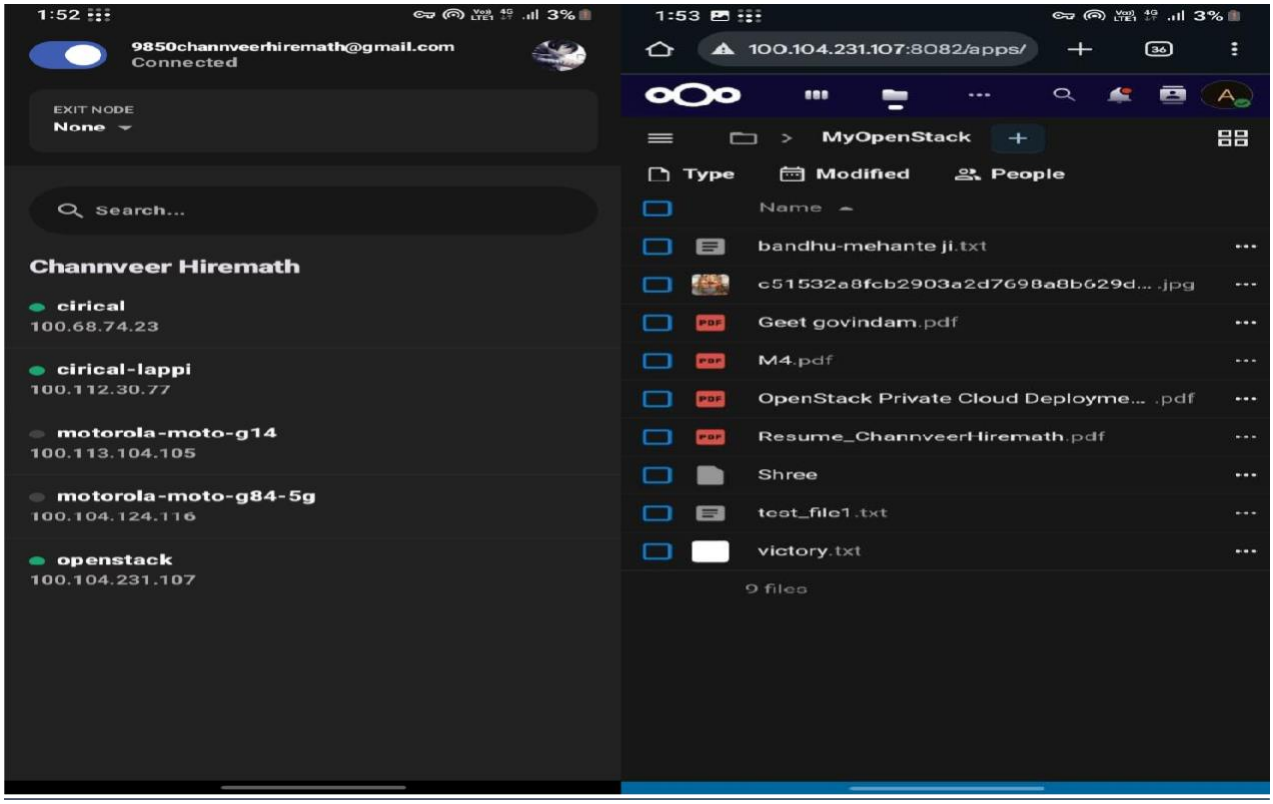
This ensured that Nextcloud would accept requests from VPN IP addresses.

Mobile Access Setup

Steps performed on Android device:

1. installed Tailscale app from Play Store.
2. logged in using the same Google account.
3. enabled VPN connection.
4. opened mobile browser.
5. accessed:

http://100.104.231.107:8082



Global Access Test

Mobile Test:

1. nextcloud login page appeared successfully.
2. logged in using credentials.
3. uploaded a file from mobile device.
4. verified in OpenStack Dashboard on PC.
5. file appeared in Swift container.

This confirmed successful remote access from mobile network.

WAN Test:

1. shared VPN access with a friend on a different network.
2. friend installed Tailscale and logged in with shared account.
3. connected to VPN successfully.
4. accessed cloud using provided credentials.
5. Successfully viewed and uploaded files.

This confirmed external access over WAN through secure VPN tunnel.

SUCCESS!!! The private cloud is now globally accessible through encrypted tailscale VPN connectivity while maintaining secure authentication.

15. Implementing Storage Redundancy & Disaster Recovery

After confirming that compute, networking, and object storage were operational, I proceeded to validate storage redundancy and disaster recovery using Cinder block storage volumes.

Creating Redundancy Volume

1. I created a new 1GB test volume for redundancy validation:

   openstack volume create --size 1 redundancy-data-final

Status: AVAILABLE

This confirmed successful volume creation.

Writing Test Data

1. to test persistence, i created a temporary server:

   openstack server create --image cirros --flavor m1.tiny --network demo-net Setup-Server

2. then i attached the new volume:

   openstack server add volume Setup-Server redundancy-data-final

3. Inside the server console:

   sudo mkfs.ext4 /dev/vdb

   sudo mkdir -p /mnt/data

   sudo mount /dev/vdb /mnt/data

   echo "Redundancy-sr" | sudo tee /mnt/data/message_data.txt

4. after writing the data:

   sudo umount /mnt/data

   openstack server remove volume Setup-Server redundancy-data-final

   This detach the volume safely.

The disaster

While attempting to create a new test VM, the cloud ran out of RAM, causing instability.

Instead of creating a new instance, I reused an existing stopped instance:

Instance : Mobile-Backend

during further operations, the dashboard became unreachable due to a network crash.

Network Recovery

1. to restore network functionality from the terminal:

   sudo ip link set enp0s8 up

   sudo ip addr add 192.168.56.250/24 dev enp0s8

this restored connectivity and brought services back online.

data survived even after crash

1. I reattached the test volume to the existing server:

   openstack server add volume Mobile-Backend redundancy-data-final

2. inside the server console:

   sudo mkdir -p /mnt/vict

   sudo mount /dev/vdb /mnt/vict

3. to verify stored data:

   cat /mnt/vict/message_data.txt

Output: Redundancy -sr

the test file was intact……

data survived:

1. deletion of original compute instance
2. failure of intermediate server attempts
3. complete network crash
4. reattachment to a different server (Mobile-Backend)

this validated that block storage volumes are independent of compute instances and maintain data persistence even in or on during the time of failures.

PTO….

16. Deployment of Windows Server 2012 Virtual Instance on OpenStack

After validating Linux-based instances, I proceeded with deployment of a Windows Server 2012 virtual machine inside OpenStack.

Downloading Windows Server Image

From the official Cloudbase website:

source: https://cloudbase.it/windows-cloud-images/

KVM	QCOW2	March 21, 2017	0.9.11	17e98010014eb8e705515d5302d5922e3e3723f4

I downloaded the Windows Server 2012 .qcow2

Transferring Image to OpenStack Host

1. after downloading the image on my local system, I copied it into the OpenStack virtual machine using

```
scp "C:\Users\User\Downloads\windows2012.qcow2" pack@192.168.56.250:/home/pack/
```

this transferred the image file to the openstack host.

Creating OpenStack Image

1. next, I created a Glance image from the uploaded qcow2 file

```
openstack image create --container-format bare --disk-format qcow2 \
  --file /home/pack/windows2012.qcow2 \
  --public \
Windows-Server-2012
```

this registered the Windows Server image in the openstack image service.

Creating Custom Flavor

since Windows Server requires more resources than Cirros, I created a custom flavor:

```
openstack flavor create --id 2 --ram 4096 --disk 40 --vcpus 2 m1.medium
```

this flavor provides:

1. 2 vCPUs
2. 4GB RAM
3. 40GB Disk

Configuring Security Group for RDP Access

By default, the security group blocks all inbound traffic.

1. to allow Remote Desktop access (RDP), I added a rule:

   openstack security group rule create --proto tcp --dst-port 3389 default

This enabled inbound traffic on port 3389.

Creating Windows Instance

1. I then launched the Windows Server instance:

   openstack server create --flavor m1.medium \

   --image Windows-Server-2012 \

   --network public1 \

   --security-group default \

   My-Windows-Server

After creation, I opened the OpenStack dashboard and accessed the instance console.

the Windows Server boot screen appeared, confirming that the image was correctly deployed.

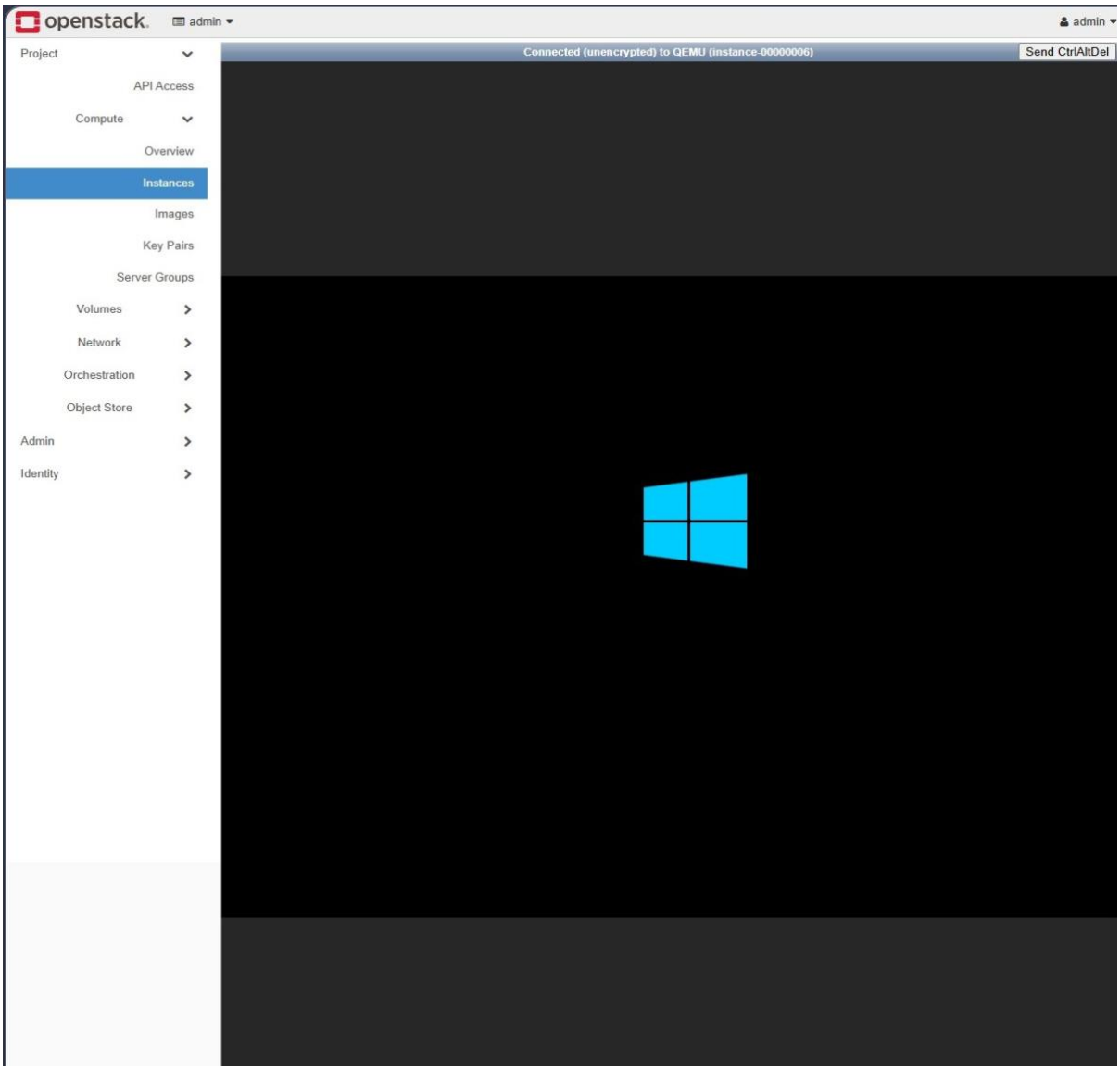


Fig: windows boot

Resource Limitation Issue

However, due to hardware resource constraints, the windows server instance could not complete booting.since the windows server flavor required 4GB RAM, there was insufficient memory available for the instance to boot successfully.

because of this limitation, the server instance could not be fully started in the current hardware setup.

This confirmed that the image deployment was correct, but physical hardware limitations prevented successful execution.